

BÁO CÁO ĐỒ ÁN MÔN HỌC

ỨNG DỤNG XỬ LÝ ẢNH TRONG NHẬN DẠNG VÀ PHÂN LOẠI CHO ROBOT WAFER

Hệ thống thị giác máy phát hiện đĩa và mô phỏng lập kế hoạch gấp-phân loại trên nền tảng Jetson

Trường: Đại học Công nghệ Kỹ thuật TP.HCM **Khoa:** Cơ khí Chế tạo Máy **Bộ môn:** Cơ Điện tử **Học phần:** Đồ án môn học Hệ thống Cơ Điện tử **Giảng viên hướng dẫn:** TS. Nguyễn Xuân Quang **Sinh viên thực hiện:** Phạm Ngọc Lan Thanh **MSSV:** 22146160 **Thời gian:** Học kỳ 1 - Năm học 2025-2026

Tóm tắt

Kính thưa Thầy, báo cáo trình bày hệ thống xử lý ảnh hoàn chỉnh cho **Robot Wafer** theo phương pháp **Perception-Measurement-Planning-Visualization**. Hệ thống thực hiện nhận dạng wafer (đĩa bán dẫn hình tròn), đo kích thước thực tế (mm) thông qua hiệu chuẩn camera, phân loại theo kích thước và mô phỏng quỹ đạo gấp robot trên nền tảng NVIDIA Jetson. Với cấu hình camera **Eye-to-Hand**, hệ thống áp dụng thuật toán **Hough Circle Transform** (ưu tiên) và **YOLOv8** (tùy chọn) để đảm bảo xử lý thời gian thực trên phần cứng nhúng.

Từ khóa: Robot Wafer, Hough Circle Transform, YOLOv8, hiệu chuẩn camera, pixel-to-mm, phân loại kích thước, Bezier path planning, NVIDIA Jetson, xử lý ảnh thời gian thực.

1. Giới thiệu và bài toán

1.1 Bối cảnh

Trong ngành công nghiệp bán dẫn, việc xử lý và phân loại wafer (đĩa silicon) đòi hỏi độ chính xác cao và tốc độ xử lý nhanh. Robot gấp wafer tự động cần khả năng nhận dạng vị trí, kích thước và phân loại wafer một cách chính xác để tối ưu hóa quy trình sản xuất.

1.2 Mục tiêu đồ án

Xây dựng hệ thống thị giác máy tính hoàn chỉnh bao gồm:

- **Nhận dạng:** Phát hiện wafer (vật thể hình tròn) trong khung hình camera
- **Đo lường:** Chuyển đổi kích thước pixel sang milimét thông qua hiệu chuẩn camera
- **Phân loại:** Xác định kích thước wafer (Small/Medium/Large) dựa trên đường kính thực
- **Lập kế hoạch:** Sinh quỹ đạo robot gấp mượt mà bằng đường cong Bezier
- **Mô phỏng:** Trực quan hóa quá trình hoạt động trong giao diện 2D

1.3 Phạm vi thực hiện

Đồ án tập trung vào thuật toán và phần mềm, mô phỏng trên nền tảng NVIDIA Jetson mà không yêu cầu phần cứng robot thực tế. Chuỗi xử lý: **thu ảnh → phát hiện → hiệu chuẩn → phân loại → phân tích không**

gian → lập kế hoạch quỹ đạo → trực quan hóa.

2. Kiến trúc hệ thống theo luồng thuật toán

2.1 Luồng xử lý chính

Hệ thống được mô tả theo pipeline: **Camera → Detector (Hough/YOLO) → Calibration (Pixel→mm) → Classifier → Spatial Analysis → Planner (Bezier) → UI**. :contentReference[oaicite:2]{index=2}

2.2 Thu nhận ảnh và ràng buộc thời gian thực

Khung hình được thu qua GStreamer, tối ưu ở độ phân giải **480×360** và mục tiêu **60 fps** nhằm giảm tải xử lý cho thiết bị nhúng. :contentReference[oaicite:3]{index=3}

3. Phương pháp và thuật toán cốt lõi

3.1 Phát hiện hình tròn bằng Hough Circle Transform (mặc định)

3.1.1 Tiền xử lý

Ảnh đầu vào được chuyển sang thang xám và làm mượt bằng Gaussian blur (kernel 9×9) để giảm nhiễu trước khi chạy Hough. :contentReference[oaicite:4]{index=4}

3.1.2 Hough Gradient và tham số

Hệ thống sử dụng biến thể Hough Gradient, với các tham số thực nghiệm (có thể tinh chỉnh theo điều kiện ánh sáng/bối cảnh):

- `minDist = 50 px`
- `param1 = 50, param2 = 30`
- dải bán kính phát hiện **10–100 px** :contentReference[oaicite:5]{index=5}

3.1.3 Ưu/nhược điểm và lý do chọn

- Ưu điểm: chạy nhanh trên CPU, không cần dữ liệu huấn luyện, kết quả mang tính tất định và dễ giải thích, footprint bộ nhớ thấp. :contentReference[oaicite:6]{index=6}
- Hạn chế: nhạy với ánh sáng, cần tinh chỉnh tham số, giảm chất lượng khi nền phức tạp. :contentReference[oaicite:7]{index=7}

Nhận xét học thuật: với ràng buộc tài nguyên và yêu cầu thời gian thực trên thiết bị nhúng, Hough Transform là lựa chọn hợp lý để ưu tiên tính ổn định/diễn giải được thay vì độ “mạnh” theo dữ liệu.

3.2 Đo lường kích thước thực bằng hiệu chuẩn 1 mốc (Single-reference Calibration)

Sau khi có bán kính (px), hệ thống chuyển đổi sang milimet theo hệ số tỉ lệ:
$$\text{mm_per_pixel} = \frac{\text{real_size_mm}}{\text{measured_pixels}}$$
 Trong đó “real_size_mm” lấy từ

vật chuẩn (ví dụ đĩa 50 mm), cho phép chuyển đổi hai chiều pixel $\leftrightarrow$ mm. :contentReference[oaicite:8]{index=8}

Ý nghĩa: bước này biến bài toán “nhìn thấy” (perception) thành “đo được” (measurement), tạo tiền đề cho phân loại kích thước dựa trên đại lượng vật lý thay vì ngưỡng pixel phụ thuộc khoảng cách/cấu hình camera.

3.3 Phân loại kích thước theo luật (Rule-based Classification)

Hệ thống chia 3 lớp kích thước dựa trên **bán kính thực (mm)**:

- Small (5 cm): < 30 mm
- Medium (7 cm): $30\text{--}42.5$ mm
- Large (10 cm): > 42.5 mm

Ngưỡng được đặt theo midpoint giữa các mức mục tiêu (30 mm và 42.5 mm). :contentReference[oaicite:9]{index=9}

Điểm nhấn thuật toán: phân loại theo luật có ưu thế “đúng–sai rõ ràng”, dễ kiểm chứng và phù hợp khi số lớp ít, đặc trưng hình học đơn giản.

3.4 Suy luận không gian mức cơ bản (Spatial Reasoning)

Hệ thống có cơ chế phát hiện chồng/nhóm dựa trên khoảng cách tâm:

- nếu khoảng cách Euclid giữa hai tâm $<$ ngưỡng (ví dụ 20 px) thì xem như có quan hệ “stack/nhóm”. :contentReference[oaicite:10]{index=10}

Bình luận: đây là suy luận không gian dạng heuristic, đủ cho mục tiêu minh họa quan hệ tương đối giữa các đĩa; đồng thời giữ chi phí tính toán thấp.

3.5 Lập kế hoạch quỹ đạo bằng đường cong Bezier bậc hai (Quadratic Bezier)

Để tạo chuyển động “mượt” từ vị trí robot đến mục tiêu, hệ thống sinh quỹ đạo theo: $[B(t)=(1-t)^2P_0 + 2(1-t)tP_1 + t^2P_2, \text{quad } t \in [0,1]]$ trong đó (P_0) là điểm xuất phát, (P_2) là mục tiêu, (P_1) là điểm điều khiển. :contentReference[oaicite:11]{index=11}

Điểm điều khiển được lấy từ trung điểm và cộng độ lệch vuông góc (hệ số 0.3) để tạo cung cong; đầu ra là khoảng 50 waypoint (x,y). :contentReference[oaicite:12]{index=12}

Ý nghĩa: Bezier bậc hai cân bằng giữa độ mượt và độ đơn giản, phù hợp mô phỏng/giảng dạy mà không cần giải bài toán tối ưu phức tạp.

4. Vì sao không triển khai YOLO trong cấu hình phần cứng hạn chế (nhấn mạnh)

Hệ thống có mô-đun YOLOv8 Nano (tùy chọn) với tối ưu TensorRT (FP16), nhưng đi kèm nhiều yêu cầu phụ thuộc và tài nguyên: CUDA-capable GPU, thư viện [ultralytics/torch/torchvision](#) và runtime TensorRT. :contentReference[oaicite:13]{index=13}

Trong khi đó, cấu hình mục tiêu phổ biến là **Jetson Nano 4GB (tối thiểu)**. :contentReference[oaicite:14]{index=14} Trên cấu hình này, việc triển khai YOLO thường gặp các bất lợi sau (trong bối cảnh dự án ưu tiên ổn định thời gian thực và đơn giản triển khai):

1. **Chi phí bộ nhớ và phụ thuộc phần mềm lớn:** YOLO yêu cầu stack PyTorch + Ultralytics + TensorRT, làm tăng độ phức tạp cài đặt và footprint bộ nhớ so với OpenCV/Hough. :contentReference[oaicite:15]{index=15}
2. **Tài tính toán và quản trị tài nguyên:** mô tả hiệu năng tham chiếu cho YOLOv8+TensorRT cho thấy mức GPU usage cao và bộ nhớ khoảng ~1.2GB (trên cấu hình mạnh hơn), trong khi Hough trên Nano có footprint ~800MB theo cấu hình hiện tại. :contentReference[oaicite:16]{index=16}
3. **Không tương xứng mục tiêu học thuật của bài toán:** đối tượng là hình tròn tương đối "dễ" về hình học, nên Hough + hiệu chuẩn + luật phân lớp đã đáp ứng yêu cầu thuật toán (phát hiện–đo–phân loại–lập kế hoạch) mà không cần năng lực biểu diễn của mạng sâu. :contentReference[oaicite:17]{index=17}

Kết luận của lựa chọn thiết kế: Thưa thầy, vì ràng buộc phần cứng (đặc biệt là Nano 4GB) và ưu tiên tính ổn định, hệ thống chọn Hough Circle Transform làm phương án mặc định; YOLO được giữ ở vai trò tham chiếu kiến trúc/so sánh, thay vì triển khai bắt buộc.

5. Thảo luận về tính đúng đắn và độ phức tạp (mức khái quát)

- **Hough Circle Transform:** độ phức tạp phụ thuộc không gian tham số và số điểm biên; giảm độ phân giải (480×360) và tiền xử lý blur giúp thực thi thực tế trên thiết bị nhúng. :contentReference[oaicite:18]{index=18}
- **Hiệu chuẩn 1 mốc:** đơn giản, dễ tái lập; đánh đổi là không mô hình hóa méo lens/biến dạng phối cảnh đầy đủ. :contentReference[oaicite:19]{index=19}
- **Bezier bậc hai:** tính toán nhẹ, sinh waypoint trực tiếp; phù hợp mô phỏng quỹ đạo trong mặt phẳng. :contentReference[oaicite:20]{index=20}

6. Kết luận

Báo cáo đã trình bày chuỗi thuật toán từ thị giác máy đến lập kế hoạch quỹ đạo trong một kiến trúc mô-đun: **Hough phát hiện hình tròn → hiệu chuẩn pixel-to-mm → phân loại theo luật → suy luận không gian heuristic → quỹ đạo Bezier bậc hai**. Các lựa chọn thuật toán nhấn mạnh tính giải thích được, chi phí tính toán thấp và phù hợp phần cứng nhúng. Đồng thời, mô-đun YOLOv8 được ghi nhận như một hướng thay thế, nhưng không phải lựa chọn triển khai tối ưu khi mục tiêu là vận hành ổn định trên cấu hình hạn chế (ví dụ Jetson Nano 4GB). :contentReference[oaicite:21]{index=21}

Tài liệu tham khảo (theo tóm tắt dự án)

1. OpenCV Documentation – Hough Circle Transform. :contentReference[oaicite:22]{index=22}
2. Ultralytics Documentation – YOLOv8. :contentReference[oaicite:23]{index=23}
3. Duda, R. O., & Hart, P. E. (1972). Use of the Hough Transformation to Detect Lines and Curves. :contentReference[oaicite:24]{index=24}
4. Redmon, J., et al. (2016). You Only Look Once: Unified, Real-Time Object Detection. :contentReference[oaicite:25]{index=25}

5. Jocher, G., et al. (2023). Ultralytics YOLOv8. :contentReference[oaicite:26]{index=26}